

Stateless Decoupler

Making a WordPress/WooCommerce store horizontally scalable on Kubernetes

Strip every stateful component out of the WordPress pod so the application tier becomes disposable — then let Kubernetes scale it on a signal that reflects what users actually feel.

Architecture & End-to-End Demo Guide

Local stack: k3d / K3s · WordPress 6.6 · PHP 8.2-FPM · MySQL 8 · Redis 7.2 · MinIO

Load generation: k6 · Observability: Prometheus, Grafana, custom React dashboard

All figures in this document are from measured runs on this stack, not estimates.

1. The problem, and why I built this

Most WordPress and WooCommerce store owners run on shared hosting. When a product goes viral or an ad lands, they cannot simply provision dedicated infrastructure in the minutes that matter — it is not operationally or financially feasible for them. The traffic arrives, the store slows to a crawl or falls over, and the spike converts into lost revenue instead of sales.

The usual defence is to over-provision: pay year-round for a server sized to a spike that happens three times a year. That is expensive and still fails if the spike is bigger than the guess.

The real blocker is that a stock WordPress install is **stateful**. Sessions, uploaded media, and store data all live on the same machine that renders the page. That coupling makes horizontal scaling unsafe: add a server and it cannot see the media uploaded to its sibling; remove one and you destroy whatever was sitting on its disk. You cannot autoscale something you cannot safely duplicate or destroy.

So I stripped the state out. Media moved to MinIO (S3-compatible object storage), the store's data moved to MySQL, and Redis took over caching, each running as its own service in the cluster rather than living inside the WordPress pod. What remains in the pod is pure computation.

THE OUTCOME

Every store pod becomes a highly disposable unit. It can be multiplied during a spike and removed once it passes — so infrastructure is sized to *current* demand rather than to the worst case, and no user loses their session, cart, or uploads when a pod disappears.

2. Core idea: separate computation from state

Everything follows from one rule: **a WordPress pod is allowed to hold nothing that cannot be thrown away**. If that holds, Kubernetes is free to create and destroy pods at will, which is precisely what autoscaling requires.

State	Now lives in	Mechanism
Uploaded media	MinIO (S3)	<code>humanmade/s3-uploads</code> , pointed at MinIO by a must-use plugin that overrides the S3 client endpoint
Object cache	Redis	<code>redis-cache</code> plugin installing the <code>object-cache.php</code> drop-in
Shopping session / cart	Redis	A custom <code>WC_Session_Handler_Redis</code> swapped in via WooCommerce's <code>woocommerce_session_handler</code> filter — see §4
Products, orders, users	MySQL	Shared database, single source of truth for every replica
Login / authentication	Nothing (client-side)	Signed cookies validated against salts shared by every pod via ConfigMap

ONE NUANCE WORTH STATING PRECISELY

"Session management" in WordPress means two different things, and conflating them invites an awkward follow-up question:

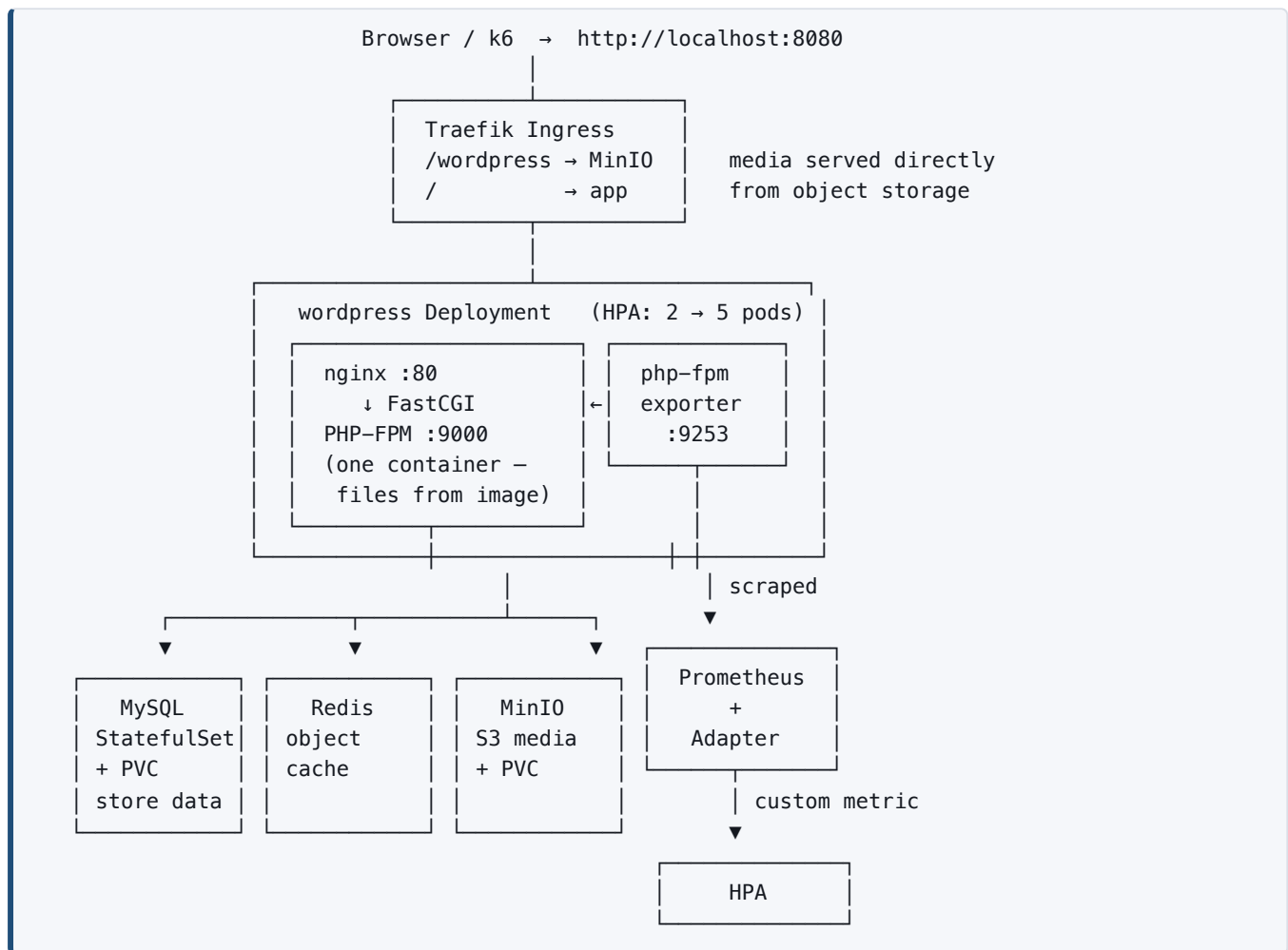
- **Login state** is not a server-side session at all. WordPress core authenticates with **signed cookies** validated against salts that every pod shares, so there is nothing to store and nothing to move.
- **The shopping session** — cart contents, applied coupons, customer details — *is* real server-side state. WooCommerce stores it in MySQL by default; here it has been moved to **Redis**.

PHP's native `$_SESSION` mechanism is configured to use Redis too, but nothing in this stack calls `session_start()`, so it holds no keys. The state that actually matters commercially is the cart, and that is in Redis.

THE PROPERTY THAT MAKES ALL OF THIS WORK

The pod holds no user state at all — not on disk, not in memory, nowhere. Login is a signed cookie, the cart is in Redis, media is in MinIO, and everything durable is in MySQL. That is precisely why a pod can be deleted mid-session without anyone noticing, which Demo 2 proves live by destroying every pod while a cart is open.

3. Architecture



Pod composition

Each WordPress pod runs two containers — no init container, no shared volume:

- **wordpress** — **nginx** and **PHP-FPM** together, both reading WordPress straight from the image layer. **nginx** serves static assets and proxies PHP over **FastCGI**; **PHP-FPM** sizes its own worker pool at startup. See §8 for why these were merged.
- **php-fpm-exporter** — reads the FPM status page over **FastCGI** and republishes it as **Prometheus** metrics. This is what makes app-aware autoscaling possible.

Why MySQL, Redis and MinIO are single replicas

This is deliberate, not an oversight. They are the fixed backing services; the whole point is that the *application tier* is what scales. Collapsing statelessness down to just the app tier is the experiment. In production each of these would need its own HA story — and MySQL is the honest ceiling of this design, since every replica opens its own connection pool against one server.

4. Moving the shopping session out of MySQL

WooCommerce ships `WC_Session_Handler`, which writes every shopper's cart to the `wp_woocommerce_sessions` MySQL table. Redis sits in front of it only as an object cache, so MySQL stays authoritative and absorbs a write on *every cart mutation, from every replica, against a single database*. On a store that scales its application tier to five pods, that is exactly the wrong thing to centralise: the tier scales out, and all of it converges on one database connection pool.

WooCommerce exposes a `woocommerce_session_handler` filter that swaps the storage class outright, so this replaces the backend rather than layering another cache over it:

```
add_filter( 'woocommerce_session_handler', function ( $handler_class ) {
    class WC_Session_Handler_Redis extends WC_Session_Handler {
        // get_session() / save_data() / delete_session() /
        // update_session_timestamp() / cleanup_sessions()
    }
    return 'WC_Session_Handler_Redis';
} );
```

Implemented in `manifests/mu-plugins/wc-redis-sessions.php`. Design decisions worth defending:

Decision	Reasoning
Keys as <code>wc_session:<id></code> with a TTL	Expiry becomes Redis's job. WooCommerce's scheduled <code>cleanup_sessions()</code> sweep becomes a no-op, removing a recurring bulk <code>DELETE</code> against MySQL.
Falls back to the stock MySQL handler on any Redis error	Losing Redis should mean slower carts, not a store that cannot take orders. Every overridden method delegates to <code>parent::</code> if the connection or command fails.
<code>pconnect()</code> rather than <code>connect()</code>	The socket persists across requests handled by the same PHP-FPM worker; otherwise every cart operation pays a fresh TCP handshake.
Class declared inside the filter callback	mu-plugins load long before WooCommerce, so referencing <code>WC_Session_Handler</code> at file scope would be a fatal error. Declaring it inside the callback also lets <code>class_exists()</code> trigger WooCommerce's autoloader for the parent.

VERIFIED ON THIS STACK

After adding one product to a cart:

```
session handler class      : WC_Session_Handler_Redis
wc_session:* keys in Redis : 1      (was 0)
rows in MySQL sessions table : 2      (unchanged – no write occurred)
key TTL                    : 172783s (~48h, WooCommerce's own expiration)
stored payload             : a:8:{s:4:"cart";...product_id;i:16...}
```

Every WordPress pod was then deleted simultaneously. The replacement pods served the identical cookie and the cart still contained its item.

5. Autoscaling on a signal that means something

The default way to autoscale is CPU, and it is the naive choice for PHP-FPM. Each PHP-FPM worker handles exactly one request at a time. When every worker is occupied, further requests sit in the listen backlog and users wait — **whether or not those workers are burning CPU**. A pod blocked on a slow database or a stalled object store is 100% saturated at near-zero CPU, and a CPU-based HPA sees an idle pod and does nothing.

THIS IS NOT HYPOTHETICAL — IT HAPPENED ON THIS STACK

When MinIO was unreachable, every request that touched media blocked on a dead endpoint. Measured result: **27 second page loads while CPU sat at 10%** and the HPA took no action. CPU was not merely slow to react; it was blind.

So the autoscaling signal was moved up the stack to worker saturation:

```
PHP-FPM status → php-fpm-exporter (sidecar)
                → Prometheus (ServiceMonitor, 10s scrape)
                → recording rule: phpfpmp_worker_utilization_ratio
                → Prometheus Adapter
                → custom.metrics.k8s.io → HorizontalPodAutoscaler
```

The HPA now evaluates three metrics and takes the largest resulting replica count:

Metric	Target	Role
phpfpmp_worker_utilization	750m (75%)	Primary — reflects user-facing capacity
CPU	70%	Backstop
Memory	80%	Backstop

The 75% threshold is deliberate rather than 100%: a new pod takes roughly 40 seconds to start serving, so scaling must begin while headroom still exists.

Sizing the worker pool from the memory limit

Hardcoding `pm.max_children` is a classic PHP-FPM failure mode. Set it too high for the memory available and the pool forks past the cgroup limit under load, so the kernel OOM-kills the container mid-request instead of PHP-FPM gracefully queuing. Each pod therefore computes its own pool size at startup from its actual memory limit:

```
max_children = (memory_limit - reserved_overhead) / avg_worker_rss
              = (768Mi - 128Mi) / 48Mi
              = 13 workers
```

MEASURED EFFECT

The stock image ships `pm.max_children = 5` regardless of how much memory the pod is given. Deriving it correctly yields **9 workers — 80% more concurrency per pod**, with no extra infrastructure. Verified degradation: a 1Gi pod computes 20 workers; a starved 128Mi pod computes 1 rather than a value that would guarantee an OOM kill.

The same formula is mirrored in the Prometheus recording rule, so the autoscaling metric's denominator can never drift from the pool size the pod actually configured.

6. End-to-end showcase

Three demos, in order. The first shows the system scaling; the second proves *why* scaling is safe; the third is the failure mode that justifies the whole metrics pipeline. Every step below has been executed on this stack.

Preparation (before the audience is watching)

```
# 1. Bring the stack up (idempotent; safe to re-run)
./scripts/bootstrap.sh

# 2. Enable app-aware autoscaling
./manifests/monitoring/setup-all.sh
helm upgrade --install prometheus-adapter \
  prometheus-community/prometheus-adapter \
  -n monitoring -f configs/grafana/prometheus-adapter-values.yaml --wait
kubectl apply -f manifests/monitoring/php-fpm-metrics.yaml

# 3. Confirm the custom metric is live (must print a value, not an error)
kubectl get --raw \
  "/apis/custom.metrics.k8s.io/v1beta1/namespaces/wordpress/pods/*/phpfpm_worker_utilization"

# 4. Start the dashboard
cd dashboard/server && npm install && node server.js      # port 4000
cd dashboard/web    && npm install && npm run dev        # port 5173
```

Open `http://localhost:5173`, and keep a terminal visible running:

```
watch -n2 kubectl get hpa,pods -n wordpress -l app=wordpress
```

TWO THINGS THAT WILL EMBARRASS YOU IF UNCHECKED

Start at 2 pods. If a previous run left the deployment at 5, the scale-out has nothing to show. Scale-down takes 5 minutes (Kubernetes' default stabilization window), so wait it out before presenting.

Start tests from the dashboard's Run test button, not from a terminal. The charts only plot runs the dashboard itself launches; a terminal-launched k6 writes to its own file and the graphs sit at "Waiting for data..."

1 Demo 1 — Autoscaling under a traffic spike

In the dashboard's cluster panel, point out **Cluster max concurrent PHP requests**. At two pods it reads 26 — that is `pm.max_children` (13) × 2 pods, and it is the honest ceiling of the application tier.

WHAT TO SAY

"Right now the store can process twenty-six PHP requests simultaneously. Everything beyond that queues. I'm about to send forty concurrent users at it — well past what it can absorb."

Select `wordpress-scale-out.js` and press **Run test**. Then narrate:

Elapsed	What happens on screen
0–30s	VUs ramp to 40; latency climbs as requests queue behind busy workers
~40s	Worker utilization crosses 750m; the HPA raises its desired replica count
~60s	New pods appear as <code>Init:0/1</code> — scale-out has begun, unprompted
~100s	Pods reach <code>2/2 Running</code> ; the capacity tile climbs
~140s	Five pods running; capacity tile reads 65
140s+	Latency falls, throughput rises, error rate stays flat

THE CLOSER

"Nobody touched anything. The application reported it was saturated, Kubernetes added capacity, and latency recovered. When traffic drops it will release those pods again — so this store is paying for peak capacity only during the peak."

2 Demo 2 — The store survives losing the pod you are using

This is the demo that actually proves decoupling, and it is the most persuasive thing in the project: **log in, then destroy the exact pod that logged you in.**

```
# Log in and keep the session cookie
curl -s -c /tmp/cookies.txt http://localhost:8080/wp-login.php -o /dev/null
curl -s -b /tmp/cookies.txt -c /tmp/cookies.txt -X POST http://localhost:8080/wp-login.php \
  --data-urlencode "log=admin" --data-urlencode "pwd=admin" \
  --data-urlencode "wp-submit=Log In" --data-urlencode "testcookie=1" \
  --data-urlencode "redirect_to=http://localhost:8080/wp-admin/" -o /dev/null

# Confirm the session works
curl -s -b /tmp/cookies.txt http://localhost:8080/wp-admin/ | grep -o "Howdy, admin"

# Identify which pod actually served that login
for p in $(kubectl get pods -n wordpress -l app=wordpress -o name); do
  n=$(kubectl logs -n wordpress $p -c nginx --tail=2000 | grep -c 'POST /wp-login.php')
  echo "$p -> $n"
done

# Destroy that specific pod
kubectl delete pod -n wordpress <the-pod-that-served-the-login>

# Same cookie, pod now gone
curl -s -b /tmp/cookies.txt http://localhost:8080/wp-admin/ | grep -o "Howdy, admin"
```

VERIFIED RESULT ON THIS STACK

After deleting the pod that served the login, the identical cookie still returns **HTTP 200** and **"Howdy, admin"**. The session is untouched, a replacement pod joins automatically, and uploaded media still loads because it was never on that pod in the first place.

The stronger version: destroy every pod with a cart open

Login survival alone can be waved away as "that's just a cookie." The shopping cart cannot — it is genuine server-side state, and it is the thing a store actually loses money on. So put an item in a cart, then delete every

WordPress pod at once:

```
# Put a product in the cart (use any published product ID)
curl -s -c /tmp/cart.txt -b /tmp/cart.txt -L \
  "http://localhost:8080/?add-to-cart=16&quantity=1" -o /dev/null

# Prove the cart now lives in Redis, and that MySQL was never written to
kubectl exec -n wordpress deploy/redis -- redis-cli --scan --pattern 'wc_session:*'
kubectl exec -n wordpress mysql-0 -- \
  mysql -uwpuser -pwppassword wordpress -N -e \
  'SELECT COUNT(*) FROM wp_woocommerce_sessions;'

# Destroy the entire application tier
kubectl delete pod -n wordpress -l app=wordpress
kubectl wait --for=condition=ready pod -l app=wordpress -n wordpress --timeout=300s

# Same cookie, every original pod gone – cart is still there
curl -s -b /tmp/cart.txt "http://localhost:8080/?wc-ajax=get_refreshed_fragments"
```

VERIFIED RESULT ON THIS STACK

Both pods were deleted simultaneously and replaced. The identical cookie returned a cart still containing its item, the Redis key `wc_session:t_d7e9ed8c...` persisted throughout, and the MySQL sessions table count **never changed** — confirming Redis is the authoritative store rather than a cache in front of the database.

WHAT TO SAY

"That cart is real server-side state — it is not a cookie. I just deleted every single server running this store, and the shopper's cart is intact, because the cart was never on any of them. This is what makes the autoscaler safe to use: Kubernetes can remove a pod at any moment and nobody loses a sale."

WHAT TO SAY

"I logged in, and that request was served by this specific pod. I'm now deleting it — not restarting it, deleting it. I'm still logged in. On conventional hosting, destroying that machine would have taken my session and every file uploaded to it. Here it costs nothing, and that's exactly what makes autoscaling safe rather than merely possible."

For a stronger version, do this *while a load test is running* and show the error-rate chart staying flat — surviving pods absorb the traffic because the Service only routes to endpoints that are `Ready`.

3 Demo 3 — Why CPU-based autoscaling is not enough

Optional, and the most senior-sounding of the three. Break the object store and watch CPU lie:

```
kubectl scale deployment/minio -n wordpress --replicas=0      # simulate a stalled dependency
curl -o /dev/null -s -w "%{time_total}s\n" http://localhost:8080/
kubectl get hpa -n wordpress
kubectl scale deployment/minio -n wordpress --replicas=1      # restore
```

WHAT TO SAY

"Latency just went to about thirty seconds. CPU is near idle, so a CPU-based autoscaler sees a healthy, underutilized cluster and does nothing at all — while every worker is blocked and every user is waiting. That's why this HPA scales on PHP-FPM worker saturation: it's the metric that actually tracks what a user experiences."

7. Measured results

From an actual run of `wordpress-scale-out.js` (40 VUs, ~5.5 minutes) on this stack:

Measurement	Result
Checks passed	1057 / 1057 (100%)
Request failure rate	0.00%
Replicas	2 → 5, reaching 5 within ~140s
Worker pool per pod	13 (autotuned, up from 5)
Peak worker utilization	1000m (100% — fully saturated)
Utilization after scale-out completed	1000m → 407m
Homepage latency, MinIO healthy vs. unreachable	0.26s vs. 27.8s
Cart survival across destruction of all pods	survived
MySQL writes for session data after the Redis handler	0

The detail worth showing

During the run, these two samples were recorded fifteen seconds apart:

time	worker utilization	CPU	replicas
11:07:05	1000m (SATURATED)	65.6%	5
11:07:20	1000m (SATURATED)	61.2%	5

Every PHP-FPM worker was busy — users were queuing — while CPU had fallen *below* its 70% threshold. A CPU-only autoscaler reads that as "under target, nothing to do." The worker metric is what correctly held the cluster scaled out.

STATE THIS HONESTLY IF ASKED

In this particular test CPU actually crossed its threshold about 15 seconds *before* worker utilization did, because rendering WordPress pages is CPU-bound — so both signals saturate together and CPU tipped first. The worker metric's decisive advantage is not speed on CPU-bound work; it is **visibility on I/O-bound work**, where CPU stays flat and never fires at all. Claiming it always reacts first would be wrong, and the data would contradict it.

8. Making scale-out fast

On an autoscaling store, **cold start is the product**. A pod that takes 44 seconds to serve traffic is 44 seconds of the traffic spike absorbed by the pods that were already overloaded. So it was worth measuring precisely rather than guessing.

Measure first — the obvious culprit was not the problem

scheduled	06:22:35		
init-html starts	06:22:49	← 14s	sandbox + container creation
init-html ends	06:22:55	← 6s	the actual file copy
containers start	06:22:57		
Ready	06:23:19	← 22s	readinessProbe initialDelaySeconds: 20
		44s	

The initContainer copies 11,947 files totalling 212MB on every pod start, so it looks like the obvious target. It was worth **6 seconds of 44**. The single largest component was `initialDelaySeconds: 20` — Kubernetes had been told not to even *check* whether the pod was ready for twenty seconds after it already was.

THE GENERAL LESSON

A defensive "give it time to boot" delay gets set early, never revisited, and quietly becomes the dominant cost. It is invisible in dashboards because nothing is failing — the pod is simply idle, ready, and not being asked.

Two fixes

1. Replace the blind delay with a probe that asks

A `startupProbe` polling once per second gates readiness and liveness until the pod genuinely serves, so a pod that is ready in two seconds is marked ready in two seconds. The probe targets `/fpm-status` over HTTP, which traverses nginx → FastCGI → PHP-FPM and so proves both processes are up *and talking* — something an open TCP port does not.

It deliberately does **not** request a WordPress page. Tying readiness to MySQL would mean a brief database blip marks every pod unready simultaneously, turning a slow query into a total outage.

2. Merge nginx and PHP-FPM into one container

nginx previously ran as a sidecar with no WordPress files of its own, which is the only reason the initContainer and the shared `emptyDir` existed at all — 212MB copied per pod, to hand nginx files it could have simply been built with. Putting nginx in the same image removes the copy, the volume and the init container together.

The two processes are started by a short script rather than `supervisord`, for two reasons. Practically, `supervisord` is broken on this image (Alpine's python/expat skew). More importantly, a supervisor that restarts a crashed PHP-FPM *inside* the container hides the failure: the pod reports zero restarts and looks healthy while silently flapping. Kubernetes is already the supervisor, so the script does the opposite — if either process exits, it exits, and the failure surfaces as a container restart. It also forwards `SIGTERM` as `nginx -s quit` and PHP-FPM's `SIGQUIT` so in-flight requests drain.

MEASURED RESULT

	before	after	
scheduled -> Ready	44s	17s	(-61%)
container -> Ready	22s	2s	probe fix
initContainer	6s	0s	merge
files copied per pod	11,947	0	
PHP workers per pod	9	13	(merged memory budget)

The remaining 15s is kubelet sandbox creation and container start, which is host-bound — on this single-machine k3d cluster it is largely contention, not application startup.

9. Known limitations and next steps

This is a local demonstration stack. Being able to name its weaknesses is part of the story.

Limitation	Detail / what I would do next
Secrets in plaintext	Credentials sit in manifests and the auth salts are placeholders, which makes cookies cryptographically worthless. Needs generated salts delivered via Kubernetes Secrets.
MySQL is the real ceiling	Single replica, no backups; every app replica opens its own connection pool against it. Moving cart sessions to Redis (§4) removed a write on every cart mutation, but products, orders and users still converge on one server. Read replicas behind a proxy are the next bottleneck to address.
Redis is now a hard dependency for carts	With sessions authoritative in Redis, a single Redis replica holds live cart state. The handler falls back to MySQL on connection failure so the store keeps trading, but carts written while Redis was down are not reconciled back. Production needs Redis replication plus AOF, not a single pod.
Cold start floor $\approx$ 15s	Addressed in §8 (44s $\rightarrow$ 17s). What remains is kubelet sandbox creation, which is host-bound. On a real multi-node cluster, pre-pulled images and a smaller image would cut it further — pruning the AWS SDK alone would remove 3,513 files for a project that only uses S3.
Readiness does not check dependencies	The probe is an HTTP check on <code>/fpm-status</code> (§8), so it proves nginx and PHP-FPM are up and talking — but not that MySQL or Redis are reachable. That is a deliberate trade-off: making readiness depend on the database means one slow query marks every pod unready at once, converting a degradation into a full outage. The better answer is a dedicated health endpoint that reports dependency state without all pods failing in lockstep.
Host-bound performance	k3d runs every node as a container on one machine, so a heavy load test starves the same control plane trying to schedule new pods. Absolute latency numbers here reflect the laptop, not the architecture.

Repository map

manifests/	Kubernetes resources + WordPress image build
wordpress.yaml	Deployment (nginx+PHP-FPM container + exporter), Service, Ingress, HPA
wordpress-config.yaml	wp-config.php and nginx vhost ConfigMaps
wordpress.Dockerfile	Custom image: phpredis, wp-cli, plugins, nginx
fpm-autotune.sh	Derives pm.max_children from the container memory limit
start-services.sh	Runs nginx + PHP-FPM, exits if either dies
mu-plugins/	wc-redis-sessions.php (cart/session store -> Redis) s3-uploads-minio.php (media -> MinIO)
monitoring/	ServiceMonitor, recording rule, custom-metric HPA
configs/	
k6/tests/	Load profiles (steady-state and scale-out)
grafana/	Prometheus and Prometheus Adapter values
scripts/	
bootstrap.sh	Zero to running stack
install-plugins.sh	Image build, WordPress install, plugin activation
dashboard/	
server/	Express + WebSocket backend streaming k6 and cluster state
web/	React + Recharts frontend